

λ -Tune: Harnessing Large Language Models for Automated Database System Tuning

VICTOR GIANNAKOURIS, Cornell University, USA

IMMANUEL TRUMMER, Cornell University, USA

We introduce λ -Tune, a framework that leverages Large Language Models (LLMs) for automated database system tuning. The design of λ -Tune is motivated by the capabilities of the latest generation of LLMs. Different from prior work, leveraging LLMs to extract tuning hints for single parameters, λ -Tune generates entire configuration scripts, based on a large input document, describing the tuning context. λ -Tune generates alternative configurations, using a principled approach to identify the best configuration, out of a small set of candidates. In doing so, it minimizes reconfiguration overheads and ensures that evaluation costs are bounded as a function of the optimal run time. By treating prompt generation as a cost-based optimization problem, λ -Tune conveys the most relevant context to the LLM while bounding the number of input tokens and, therefore, monetary fees for LLM invocations. We compare λ -Tune to various baselines, using multiple benchmarks and PostgreSQL and MySQL as target systems for tuning, showing that λ -Tune is significantly more robust than prior approaches.

CCS Concepts: • **Information systems** → **Autonomous database administration**; **Database utilities and tools**.

Additional Key Words and Phrases: Database, Tuning, Large Language Models

ACM Reference Format:

Victor Giannakouris and Immanuel Trummer. 2025. λ -Tune: Harnessing Large Language Models for Automated Database System Tuning . *Proc. ACM Manag. Data* 3, 1 (SIGMOD), Article 2 (February 2025), 26 pages. <https://doi.org/10.1145/3709652>

1 Introduction

The performance of database management system changes dramatically as a function of various tuning choices, including settings for system configuration parameters as well as physical design choices such as indexing, sorting, or partitioning. This has motivated a large body of research on automated database system tuning. Recent work exploits machine learning to find near-optimal configurations [5, 6, 14, 24] but suffers from high training and exploration overheads. This has motivated a new line of research [12, 21], exploiting LLMs to heuristically prune the search space for tuning. Similar to human database administrators, such models leverage commonsense knowledge, extracted from text documents, to narrow the focus to tuning options that seem “reasonable”, given the tuning context. This paper presents λ -Tune [7] (LAnguage Models for Better Database Administration), a system that exploits capabilities offered by the latest generation of LLMs, including the likes of GPT-4 and Claude 3, to optimize various tuning choices for specific systems and OLAP workloads, including system parameter settings as well as physical design decisions.

λ -Tune. Prior approaches to LLM-enhanced database tuning [12, 21] parse text documents (e.g.,

Authors' Contact Information: Victor Giannakouris, Cornell University, Ithaca, NY, USA, vg292@cornell.edu; Immanuel Trummer, Cornell University, Ithaca, NY, USA, it224@cornell.edu.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 2836-6573/2025/2-ART2

<https://doi.org/10.1145/3709652>

the database manual) to extract value recommendations for specific parameters. They still need to perform an optimization stage in which hints about specific parameters are combined into complete configurations. This approach is in line with the limitations of early-stage language models such as BERT [4] and GPT-2 [15]. For those models, input and output sizes are limited to a few hundred tokens, restricting the scope of these models to settings for single parameters (rather than entire configurations). Modern LLMs such as GPT-4 support input and output sizes of hundreds of thousands of tokens. The design of λ -Tune is motivated by these advances. It exploits increased input sizes by feeding to the language model a description of all information relevant for tuning, including the workload and target system. It also exploits the increased output size by generating entire configurations, rather than hints about single parameters. As shown in our experiments, modern LLMs such as GPT-4 are typically able to map information about the workload to efficient database configuration settings. Hence, unlike prior systems, λ -Tune avoids expensive optimization steps, combining settings for single parameters. Instead, it delegates more responsibility to the language model itself.

Prompt Generation. First, λ -Tune automates the *prompt generation* step by crafting prompts tailored to the input workload (analytical SQL queries), hardware specifications, and the database system. Our approach incorporates a workload representation method that decomposes the input SQL queries into much smaller, mergeable components, called *query snippets*. As costs increase in the prompt size, minimizing monetary fees while conveying the most relevant information is challenging. We select the most informative subset of snippets to include in the prompt, given a bound on the number of prompt tokens (which are proportional to processing fees for providers like OpenAI). We formulate workload representation as a cost-based optimization problem that we solve by a transformation to integer linear programming. Using the resulting prompt, λ -Tune issues multiple calls to the LLM with a certain degree of randomization to obtain multiple candidate configurations. By running the input queries with different configurations, λ -Tune evaluates and identifies the most efficient configuration among them using the ideas discussed next.

Configuration Selection. The LLM may return configurations of varying quality. In this context, a challenge is to avoid slowdowns due to particularly bad configurations, incurred, for instance, when evaluating configurations sequentially. To tackle this challenge, we introduce a configuration selection approach that incrementally evaluates the obtained configurations in multiple rounds. Each round comes with a timeout that limits the impact of bad configurations on tuning time. On the other hand, interrupting execution repeatedly may cause redundant work. λ -Tune chooses timeouts according to a geometric progression scheme, limiting wasted work due to interruptions before the final round. At the same time, it avoids re-evaluating the same queries across multiple rounds and calculates configuration-specific timeouts, taking into account work accomplished in prior rounds. Reconfiguration overheads, e.g., index creations, may dominate query evaluation time if switching between configurations with a high frequency. Hence, λ -Tune adapts query evaluation timeouts to ensure that reconfiguration overheads are proportional to query run time.

Configuration Evaluation. Changing between different configurations can be costly, in particular if it involves index creations. This makes it challenging to keep switching overheads low during evaluations. λ -Tune minimizes these overheads by utilizing a *lazy index creation* approach, that only creates the indexes before the execution of a query that might use them, according to the referenced column. At the same time, λ -Tune optimally orders query execution according to their index creation costs using a dynamic-programming-based query scheduler, which minimizes query reconfiguration costs when switching between different configurations. Our algorithm is based on a custom cost model we built for our query scheduling needs. We prove that the principle of optimality applies to this cost model in Section 3.

We evaluate λ -Tune over Postgres and MySQL, using the Join Order Benchmark (JOB) and TPC-H as benchmarks. Our experimental evaluation illustrates λ -Tune’s robustness, outperforming prior tools for automated database system tuning, including GPTuner [12], DB-Bert [22], UDO [24], LlamaTune [9], as well as ParamTree [25]. In summary, our original scientific contributions are the following:

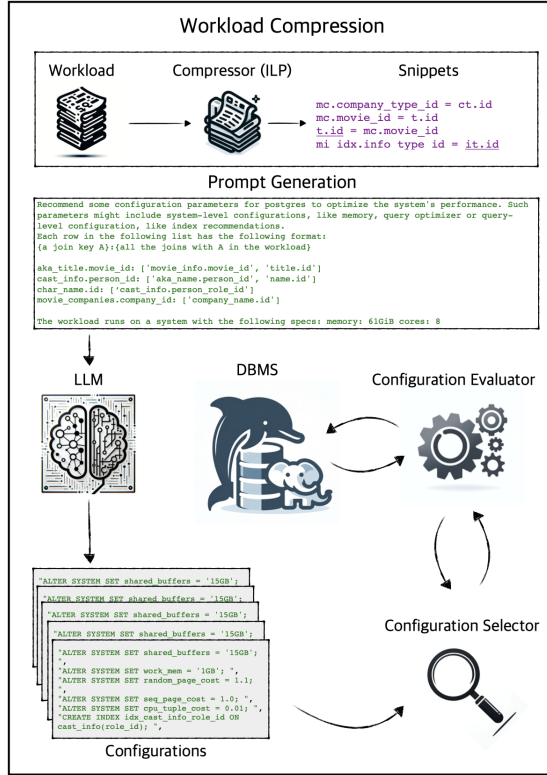
- We present λ -Tune, a framework that harnesses Large Language Models for automated, database system tuning for Online Analytical Processing (OLAP) workloads.
- We introduce three powerful components that facilitate our LLM-assisted tuning approach, including prompt engineering, configuration selection, and configuration evaluation.
- We present a thorough experimental evaluation that showcases that λ -Tune is the most robust among its competitors, by consistently identifying the configuration that achieves the best performance.

The rest of this paper is organized as follows. Section 2 presents an overview of λ -Tune, its design, and main goals. Next, in Section 3, we describe the prompt generation component, which includes our workload compression method. In Section 4, we present our configuration selection approach. Next, Section 5 presents the configuration evaluation component. Section 6 presents our experimental evaluation of λ -Tune compared to three baselines, as well as an ablation study that showcases the effectiveness of the λ -Tune’s individual components. Finally, in Sections 7 and 8 we present the related work before concluding.

2 λ -Tune

Overview. λ -Tune’s architecture is depicted in Figure 1. λ -Tune leverages LLMs to automate the tuning of database systems for OLAP workloads, ensuring to find the configuration that achieves the optimal performance, among the configurations obtained from the LLM. Existing approaches to automated database system tuning depend on the availability of training data, excessive training overheads, and tuning rounds. λ -Tune is built on the intuition that the *whole tuning task can be described as a concise prompt* and *downstreamed to an LLM* which already contains and can blend domain-specific tuning information using its pre-trained weights. While not done in the current system, this approach could easily be augmented via retrieval augmented generation, enabling the LLM to parse additional information from the Web. λ -Tune takes as input three parameters: an OLAP workload $\mathcal{W} = \{q_1, q_2, \dots, q_n\}$ consisting of n queries, a hardware specification $\mathcal{H}$ consisting of the number of cores and memory in the system, and a database system name $\mathcal{D}$. It integrates these three parameters into a prompt tailored for the given setup and obtains configurations from the LLM to optimize the performance of the target system. Optionally, the user can define a token budget $\mathcal{B}$ for the prompt generator, if they wish to restrict the API costs, otherwise, λ -Tune will try to fit as much information as possible into the prompt, according to the language model token limit.

Tuning Pipeline. Algorithm 1 presents the tuning pipeline of λ -Tune. The first step is to pass the input parameters (an OLAP workload, hardware specification, and the target database system) to the prompt generator, described in Section 3. The prompt generator will first compress the input workload, as described in Section 3.2, in order to break down the input queries into smaller text snippets that contain information about specific operators like joins or selections. Then, it will select and combine the most informative snippets for the LLM, with respect to the token budget. Next, the prompt generator transforms and embeds the compressed workload, along with the rest of the input parameters, into a prompt that describes the workload, the database system, and the hardware to the LLM. Next, it invokes the LLM k times, in order to retrieve k responses, each one including a single full configuration. The retrieved configurations will differ according

Fig. 1. λ -Tune Architecture

to the degree of randomization of the LLM, determined by the temperature. Each configuration contains a set of SQL commands, compliant with the target database. For instance, if the target system is Postgres, the configuration will typically consist of a list of “CREATE INDEX” and “ALTER SYSTEM SET \$param_name = \$value” commands. λ -Tune is designed with the assumption that some of the retrieved configurations might be disproportionately slower than the efficient ones. To handle such scenarios, we use an approach that evaluates the retrieved configurations in multiple rounds with a given per-round timeout, preventing inefficient configurations from monopolizing the whole tuning process. This approach provides *provable time guarantees* that are *bounding the tuning time by a function of the optimal execution time (among all configurations retrieved from the LLM)*, as we discuss in Section 4. Furthermore, to minimize index reconfiguration overheads during evaluation, we first associate indexes with queries that could exploit them, based on column references, and create them lazily, only before an associated query execution. To minimize index reconfiguration costs, we optimally order the queries according to their index generation costs, using a dynamic-programming algorithm presented in Section 3.

3 Prompt Generation

We describe details of λ -Tune’s prompt generation process, including the prompt template used as well as our approach for generating a compressed representation of the input workload.

Algorithm 1: λ -Tune

```

1 Function Tune( $\mathcal{W}, \mathcal{H}, \mathcal{D}, \mathcal{B}$ ):
    /*  $\mathcal{W}$ : The input workload (queries) */
    /*  $\mathcal{H}$ : The hardware specification (cores, memory) */
    /*  $\mathcal{D}$ : The database system (e.g. "Postgres", "MySQL") */
    /*  $\mathcal{B}$ : The token budget */

    /* Generate the prompt */
2   prompt = GeneratePrompt( $\mathcal{W}, \mathcal{H}, \mathcal{D}, \mathcal{B}$ )
    /* Send  $n$  API calls to the LLM */
3    $C = \text{LLM}(\text{prompt}, n)$ 
    /* Find the best configuration */
4   best = ConfigSelect( $\mathcal{W}, C, t, \text{max\_rounds}$ )
5   return best

```

3.1 Prompt Template

Listing 1 depicts our prompt template. Placeholders that are substituted anew for each tuning problem instance are surrounded by curly braces ($\{\dots\}$). The prompt template starts with general instructions about the task we expect the LLM to solve: database tuning. The instructions are fairly generic while providing examples of several tuning choices, e.g., related to indexing or memory allocation, that tend to have a significant impact on performance. The placeholder $\{\text{DBMS}\}$ is replaced by the name of the target database management system to tune (e.g., PostgreSQL or MySQL). Due to knowledge gained via pre-training, large LLMs such as GPT-4 are able to adapt the commands for configuration changes to the target system without further instructions.

The next block contains an aggregate description of the input workload. Providing SQL queries directly leads to significant costs for large workloads (since processing fees for LLMs, hosted by providers such as OpenAI, are proportional to input and output sizes). Hence, we provide a compressed representation instead, focusing only on the most important workload aspects while representing information as concisely as possible. As justified in more detail in the following, we focus on describing the join structure of the input workload in the placeholder $\{\text{COMPRESSED_WORKLOAD}\}$. The preceding sentence provides the LLM with instructions on how to interpret the following, compressed representation.

The final text block of the prompt template contains information on the hardware properties of the target system. For our use cases, we find it sufficient to include the amount of main memory and the number of CPU cores. The prompt template can be extended easily to integrate more details on the hardware.

3.2 Workload Compression

We present an approach to summarize input workloads concisely for the LLM, thereby reducing LLM-related processing fees. Our description focuses on binary relationships between different parts of the data. For instance, such relationships could describe the collocation of tables on the same machine in a distributed setting, the co-occurrences of tables in the same queries, or the connections between column pairs that appear in the same join conditions. This type of information is important for tuning decisions such as data partitioning, indexing, and replication. While our approach easily extends to each of the aforementioned properties, we specifically use it to represent

```
Recommend some configuration parameters for ${DBMS}$ to
optimize the system's performance. Parameters might
include system-level configurations, like memory,
query optimizer or physical design configurations,
like index recommendations.
```

```
Each row in the following list has the following format:
{a join key A}:{all the joins with A in the workload}
${COMPRESSED_WORKLOAD}$
```

```
The workload runs on a system with the following specs:
memory: ${MEMORY}
cores: ${CORES}
```

Listing 1. λ -Tune Prompt Template

join conditions in the current implementation. The motivation for this choice is the fact that joins tend to be among the most expensive operators. Therefore, providing information that helps to optimize the database setup for reduced join overheads is a priority.

Denote by $P \subseteq \{\langle c_1, c_2 \rangle | c_1, c_2 \in C\}$ the set of pairs of join columns from C that appear together in a join condition in the input workload. The first possible representation is simply the list of these column pairs. However, this representation is sub-optimal as it requires more space than necessary. Instead, it is preferable to merge column pairs that share at least one common column. We use a compressed representation that pairs up a specific column c_1 with each column c_2 that appears together with c_1 in a join predicate. In the prompt, we associate each line of the workload description with one column on the left-hand side, separated by a colon from a comma-separated list of associated columns on the right-hand side. By providing the LLM with instructions on representation semantics (see Listing 1), we enable the LLM to correctly interpret the workload summary.

Example 3.1. Assume that P contains the following column pairs: $\langle A, B \rangle$, $\langle A, C \rangle$, and $\langle A, D \rangle$. In a compressed representation, these three binary relationships are summarized in a single line of the prompt: $A:B, C, D$.

Even with the aforementioned compression techniques, it is still not possible to represent the full join structure of large workloads with diverse join conditions. To comply with intrinsic limits on the number of input tokens, associated with all LLMs, as well as with budget constraints of users (since processing more input is more costly), it is necessary to choose which subset of join conditions to represent in the prompt. Our selection strategy is based on the intuition that conditions associated with more expensive joins are more important. The current cost of a join gives an upper bound on how much cost can be reduced via tuning. If the LLM is unaware of the most expensive joins, it cannot effectively reduce processing overheads. Hence, we associate each join column pair $p \in P$ with a value $V(p)$ that represents the total cost associated with joins that use the corresponding condition. We calculate $V(p)$ as the sum $\sum_{j \in J(p)} EC_j$ where $J(p)$ is the set of all join operators in which join condition p is evaluated (considering the default plans chosen by the query optimizer) and EC_j the estimated processing cost, associated with join operation j (this cost can be obtained from the optimizer using corresponding EXPLAIN commands). Now, given a limit on the number of tokens used to represent the workload and values H_c , representing the number of tokens required to represent column $c \in C$, our goal is to select query snippets, in the form of join conditions, that maximize the accumulated value of join conditions conveyed to the LLM.

Table 1. Workload Compressor, ILP Constraints

Variable	Semantics
$R_{\langle c_1, c_2 \rangle} \in \{0, 1\}$	Binary variable which denotes if c_2 appears on the right-hand side of c_1 .
$L_c \in \{0, 1\}$	Binary variable which denotes whether the left-hand side column c is included in the prompt.
Constraint	Semantics
$R_{\langle c_1, c_2 \rangle} \leq L_{c_1}$	A right-hand side column c_2 can be included only if its left-hand side column is included.
$L_{c_1} \leq \sum_{\langle c_1, c_2 \rangle \in P} R_{\langle c_1, c_2 \rangle}$	A left-hand side column can be included only if at least one right-hand side column is included.
$R_{\langle c_1, c_2 \rangle} + R_{\langle c_2, c_1 \rangle} < 2$	Symmetric join snippets cannot be added together.

3.3 ILP Formulation

Picking an optimal combination of join conditions to include in the prompt under a constraint on the number of tokens is a non-trivial problem. Even without considering the possibility of compressing multiple join conditions sharing the same column, it relates to the knapsack problem (weights correspond to token consumption H and utility to the processing cost values V) which is NP-hard. Hence, we transform the problem into an integer linear programming problem (ILP) to apply corresponding software solvers.

We introduce binary decision variables L_c for each column $c \in C$, indicating whether or not the corresponding column appears on the left-hand side of a line in the prompt. Also, we introduce binary variables R_p for $p = \langle c_1, c_2 \rangle \in P$, indicating whether or not c_2 appears on the right-hand side of c_1 in a line in the prompt. Clearly, there are dependencies between the two groups of variables. If $R_{\langle c_1, c_2 \rangle}$ is set to one, indicating that c_2 appears to the right of c_1 , the associated variable L_{c_1} must be set to one as well: $L_{c_1} \geq R_p$ for all $p = \langle c_1, c_2 \rangle \in P$. Similarly, if column c appears on the left-hand side, i.e., $L_c = 1$, we can prune the search space by imposing at least one associated column on the right-hand side: $L_{c_1} \leq \sum_{\langle c_1, c_2 \rangle \in P} R_{\langle c_1, c_2 \rangle}$. Our goal is to maximize accumulated value while limiting token consumption by budget $\mathcal{B}$:

$$\begin{aligned}
 &\text{Maximize: } \sum_{p \in P} V(p) R_p \\
 &\text{Subject to: } \sum_{\langle c_1, c_2 \rangle \in P} H_{c_2} \cdot R_{\langle c_1, c_2 \rangle} + \sum_{c \in C} H_c \cdot L_c \leq \mathcal{B}
 \end{aligned}$$

To avoid double-counting pairs of join columns that are symmetric (e.g., $A:B$ versus $B:A$), we add one more constraint, avoiding redundant join conditions: $R_{\langle c_1, c_2 \rangle} + R_{\langle c_2, c_1 \rangle} < 2$. Table 1 summarizes all of the aforementioned constraints and variables.

4 Configuration Selector

The LLM may generate configurations of varying quality. Evaluating those configurations sequentially may lead to large overheads due to particularly slow configurations. Next, we describe how λ -Tune avoids this issue.

Incremental Timeouts. Algorithm 2 describes λ -Tune’s configuration selection approach. Table 2 summarizes the fields of the *ConfigMeta* objects that are used in several of the following

algorithms (during initialization, values for those fields are provided in the order in which they appear in Table 2). To avoid spending too much time evaluating bad configurations, Algorithm 2 proceeds in rounds and imposes a per-configuration timeout in each round. Initially, it is unclear what timeout allows the best configuration to finish. Hence, Algorithm 2 increases an initial timeout t gradually by multiplying the timeout with factor α in each round. Having a geometric progression for the timeout is crucial, as it guarantees that the total time spent in all previous rounds (which may be wasted if query evaluation is interrupted due to timeouts) is always proportional to the time spent in the last round (in which at least one configuration finishes executing all queries before the timeout).

Avoiding Redundancy. Evaluating the same queries with the same configuration is redundant (unless query evaluation is interrupted by timeouts). Hence, λ -Tune keeps track of the queries that were fully processed for each configuration. Completed queries are stored for each configuration in the *completedQueries* field of the *ConfigMeta* object associated with that configuration. Meta-data about configurations is generally stored in the *configsMeta* dictionary, mapping configurations to meta-data and initialized in Lines 3 to 5. When selecting queries for evaluation for a specific configuration, λ -Tune removes queries already processed (Line 20).

Best Configuration. Algorithm 2 keeps track of the best currently known configuration in the *best* variable. This variable is of type *BestConfig* featuring two fields: the *time*, indicating the execution time of the best currently known configuration, and *config*, describing the best configuration itself. As long as no candidate configurations have been fully evaluated (i.e., all queries have been fully processed with those configurations), the fields of *best* are set to default values, initialized in Line 2. The while loop from Lines 6 to 13 terminates once at least one configuration has been fully evaluated. It may seem like the first configuration to finish should be the optimal one. That, however, is not generally the case, as illustrated by the following example.

Example 4.1. Consider a workload $W = \{q_1, q_2, q_3\}$, an initial timeout $t = 1$, and two configurations c_1 and c_2 , where the times for queries q_1, q_2 , and q_3 are 1, 2, and 4 in c_1 and 5, ϵ , ϵ in c_2 . In the first three rounds (with timeouts of 1, 2, and 4 seconds respectively), c_1 will execute all of the queries, taking 7 seconds in total. However, c_2 can achieve a better time of $5 + 2 \cdot \epsilon$ seconds, despite the fact that it did not complete any query execution earlier. This means that even if c_1 finishes first, configuration c_2 is the better configuration overall.

Therefore, λ -Tune identifies the optimal configuration as follows. Once the first configuration terminates, Algorithm 2 uses a different timeout that may, in fact, be higher than the timeout at the start of the corresponding round. λ -Tune uses as timeout the execution time of the best currently known configuration (which may be updated repeatedly as other configurations finish) *minus* the time spent fully evaluating queries by the corresponding configuration (Line 19). Any configuration exceeding that timeout is guaranteed to be sub-optimal. The aforementioned timeout can only be set once the total execution time of a first configuration is known. Before that happens, the default timeout of the corresponding round applies. Therefore, once the first configuration finishes, all of the other configurations must be given the chance to finish with the new timeout. This happens in the loop from Lines 14 to 15. After that, the algorithm returns the best configuration. Note that finding good configurations earlier is preferable since it enables tighter timeouts, thereby reducing time wasted on sub-optimal configurations. Therefore, λ -Tune iterates over configurations in decreasing order of throughput (i.e., number of queries finished per time unit), assuming that the configuration with the currently highest throughput is more likely to be optimal (which is, of course, not guaranteed).

Example 4.2. Figure 2 depicts an example of our configuration selection approach. The x-axis represents execution time, and the y-axis represents the configuration ID. We assume that the

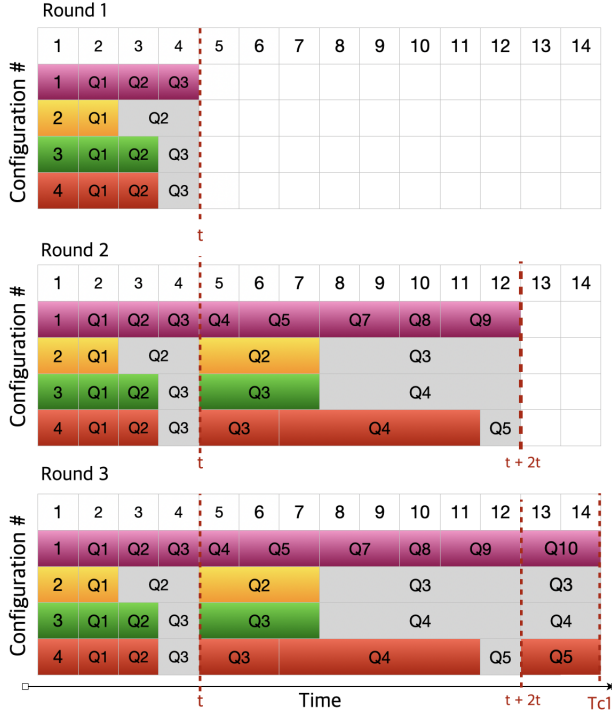


Fig. 2. λ -Tune Configuration Evaluation

timeout increases by factor $\alpha = 2$ from one iteration to the next, starting with a timeout of $t = 4$. Each configuration is represented with a different color, and a colored square represents a completed query of the configuration of the same row. Gray squares indicate that the last executed query was interrupted due to a timeout. In Round 1, Configuration ① completes three queries, Configuration ② completes one query, and gets interrupted while executing Q_2 . Configurations ③ and ④ execute two queries and get interrupted while executing Q_3 . In round two, the timeout doubles to $2 \cdot t = 8$, meaning that the second round stops after 12 time units total (taking into account the first round as well). Finally, in Round 3, Configuration ① completes all 10 queries after 14 time units total, implying now a configuration-specific timeout for each of the other configurations. None of the remaining configurations terminate within the new timeout. Thus, λ -Tune returns Configuration ① as the optimum.

Time Guarantees. So far, we have justified the choice of timeouts intuitively. Now, we provide a formal proof, showing that our timeout scheme bounds total tuning time for query evaluation as a function of the optimal configuration returned by the LLM.

THEOREM 4.3. *The total tuning time (excluding reconfiguration overheads) is in $O(k \cdot \alpha \cdot C_{best})$, where C_{best} is the execution time of the best configuration returned by the LLM, for $\alpha \geq 2$.*

PROOF. The timeout increases by factor α from one round to the next. Hence, the timeout of the final round, T_{last} , is higher than C_{best} at most by that factor: $T_{last} \leq \alpha \cdot C_{best}$. The execution time of each configuration in the last round is upper-bounded by T_{last} . Hence, the total execution time of the final round for k configurations is upper-bounded by $k \cdot T_{last} \leq k \cdot \alpha \cdot C_{best}$. However, as the

Algorithm 2: Configuration Selection

```

1 Function ConfigSelect( $W, C, t, \alpha$ ):
    /*  $W$ : The input workload */
    /*  $C$ : The input configuration sets */
    /*  $t$ : The initial timeout */
    /*  $\alpha$ : The timeout ratio */

    /* Initialize best configuration */
2  $best \leftarrow \text{BestConfig}(\infty, \text{null})$ 
    /* Initialize configuration meta-data */
3  $\text{configsMeta} \leftarrow \text{dict}()$ 
4 for  $c \in C$  do
5    $\text{configsMeta}[c] \leftarrow \text{ConfigMeta}(0, \text{False}, 0, \emptyset)$ 
    /* Until first configuration finishes */
6 while  $best.time == \infty$  do
7   for  $c \in C$  in decreasing order of throughput do
8     /* Evaluate next queries for this configuration */
9      $\text{Update}(c, W, \text{configsMeta}, t, best)$ 
10    if  $\text{configsMeta}[c].isComplete$  then
11       $\text{candidates} \leftarrow C - \{c\}$ 
12      break
13    /* Consider re-configuration overheads */
14     $t \leftarrow \max_{c \in C}(t, \text{configsMeta}[c].indexTime)$ 
15     $t \leftarrow \alpha \cdot t$ 
    /* Check whether remaining configurations are better */
16 for  $c \in \text{candidates}$  in decreasing order of throughput do
17    $\text{Update}(c, W, \text{configsMeta}, t, best)$ 
18 return  $best.config$ 

19 Procedure Update( $c, W, \text{configsMeta}, t, best$ ):
    /* Updates all relevant data structures after query evaluations */
20 if  $best.time \neq \text{inf}$  then
21    $t \leftarrow best.time - \text{configsMeta}[c].time$ 
    /* Execute only the non-executed queries */
22  $Q \leftarrow W - \text{configsMeta}[c].completedQueries$ 
23  $\text{Evaluate}(c, Q, t, \text{configsMeta})$ 
24 if  $\text{configsMeta}[c].isComplete$  then
25   /* Keep the best configuration */
26   if  $\text{configsMeta}[c].time < best.time$  then
27      $best.time \leftarrow \text{configsMeta}[c].time$ 
28      $best.config \leftarrow c$ 

```

series of timeouts forms a geometric progression with factor $\alpha \geq 2$, the accumulated time of all prior rounds is at most equal to the time of the last round. $\square$

Table 2. Field Names and Descriptions of the *ConfigMeta* Object Used in Algorithms 2 and 3

Field Name	Description
<i>time</i>	Completed query time
<i>isComplete</i>	Configuration completion flag
<i>indexTime</i>	Index creation time
<i>completedQueries</i>	Set of completed queries

Reconfiguration Overheads. We have bounded time overheads due to executing queries with different configurations. However, tuning time also depends on the time required for configuration changes. In practice, overheads for index creations tend to dominate reconfiguration overheads. Index creation overheads have the potential to dominate tuning time. For instance, starting tuning with timeouts that are fairly small, compared to index creation overheads, would lead to inefficient tuning, spending more time creating indexes than executing queries. To avoid such cases, λ -Tune takes into account index generation overheads when setting timeouts. More precisely, λ -Tune measures index generation overheads and adapts timeouts accordingly (see Line 12 of Algorithm 2).

5 Configuration Evaluator

We describe how λ -Tune evaluates configurations efficiently while minimizing re-configuration overheads.

5.1 Evaluating Configurations

Algorithm 3 shows pseudo-code for the Evaluate procedure, used in Algorithm 2. As input, this code takes a configuration c to evaluate, a set Q of queries that have not been fully processed using this configuration, a timeout t , and a dictionary mapping configurations (including c) to associated meta-data (featuring the fields introduced in Table 2 in the previous section).

The effect of executing the procedure is that the meta-data for configuration c , stored in *configsMeta*[c], gets updated with the results of the evaluation. We assume a call-by-reference model such that changes to the aforementioned data structure will be visible to the calling function after executing Evaluate.

Algorithm 3 keeps track of the time remaining for the next queries which is initialized to the input timeout in Line 2 and is updated each time that a query is processed (Line 16). The remaining time is provided as an input parameter to the Execute function, executing single queries (first parameter) with a timeout (second parameter). The Evaluate procedure automatically creates indexes that are required by the current configuration. However, it is inefficient to create all indexes associated with the configuration immediately. Due to the timeout, query evaluation may end long before the last query has finished. This means that creating indexes that are only relevant to later queries is wasteful if those queries are not executed. Note that all indexes created in this procedure are implicitly dropped once the procedure terminates (to enable us to evaluate other configurations without spurious indexes). This means that indexes that are not used would need to be re-created in the next invocation.

Therefore, Algorithm 3 creates indexes *lazily*, creating only the indexes that are potentially relevant for the next query at hand. To determine whether indexes may be relevant, λ -Tune analyzes the column and table references in those queries, creating a map I that maps queries to the set of potentially relevant indexes (this map is created in Line 6). To assess whether an index could be useful for a query, we check whether the indexed columns overlap with columns that appear in

query predicates. Before evaluating each query, λ -Tune creates the indexes possibly needed for this specific query (Line 9). At the same time, λ -Tune keeps track of index generation overheads (which are used in Algorithm 2 to update timeouts). Function `createIndexes` returns the time needed to create the corresponding indexes while, at the same time, creating them. Not all relevant indexes must be created as some of them may have been relevant for prior queries, too. Algorithm 3 keeps track of all existing indexes in the `createdIndexes` variable and subtracts existing indexes from the set of indexes to create (Line 9).

Procedure `Evaluate` terminates once the first query is interrupted due to a timeout. In that case, the flag named `complete`, included in the result returned by `Execute` (this function executes the input query), is set to false. Algorithm 3 checks for interruptions in Line 12. If queries are interrupted, it sets the `isComplete` flag of the configuration meta-data to false. If no query is interrupted, this field keeps the value of True it is initialized with (Line 4).

Note that Algorithm 3 processes queries in a carefully chosen order, implemented by the `FindOptimalOrder` function. The following subsections discuss the benefit of that function and its implementation.

5.2 Cost Model for Query Scheduling

The order in which queries are processed can have a significant impact on index creation overheads. If all queries were to be executed, index generation overheads would be independent of query order. However, taking into account possible interruptions, more likely to affect later than earlier queries, the order can impact cost. This is illustrated in the following example.

Example 5.1. Assume we process two queries, q_1 and q_2 . Each query can use only one query-specific index whose creation cost is 1 for q_1 and 5 for q_2 . Assume that both queries have the same run time and that the execution of the first query, whichever one it is, gets interrupted with a probability of 50%. Using the order $q_1 - q_2$, the expected cost for index creations is $1 + 0.5 \cdot 5 = 3.5$, i.e., we definitely must create the index for the first query but incur the cost for the second query index only with a probability of 50%. Otherwise, if using order $q_2 - q_1$, the expected cost is $5 + 0.5 \cdot 1 = 5.5$. Here, we definitely pay for creating the expensive index of the second query, whereas the index creation overheads of the first query are only paid in 50% of cases.

As shown in the example, query order matters for performance. λ -Tune uses cost-based optimization to order queries. In this subsection, we discuss the cost model it uses. We generalize from the example above. We calculate expected costs for index creations, taking into account different possibilities for the point at which query execution is interrupted due to the timeout. We simplify by assuming that an interruption after each query is equally likely. While highly simplifying, the orders resulting from this model tend to improve performance over default orders, as shown in the experiments.

Given n queries q_1 to q_n to evaluate, we assume that an interruption after each of them (or the case that all of them start execution) is equally likely, i.e., have a probability of $1/n$. Denote by $z_i(Q)$ the index creation overheads for the query i , assuming that the set of queries Q has been evaluated before (this is important since the set of indexes to create for the new query depends on which indexes were created for prior queries). Given query order i_j for $1 \leq j \leq n$, the cost for interrupting after the k -th query is given by $\sum_{1 \leq j \leq k} z_{i_j}(\{q_{i_1}, \dots, q_{i_{j-1}}\})$. Hence, the total expected execution cost is given by

$$\frac{1}{n} \cdot \sum_{1 \leq k \leq n} \sum_{1 \leq j \leq k} z_{i_j}(\{q_{i_1}, \dots, q_{i_{j-1}}\}) \quad (1)$$

Algorithm 3: Configuration Evaluation

```

1 Procedure Evaluate( $c, Q, t, configsMeta$ ):
    /*  $c$ : The input configuration */
    /*  $Q$ : The input queries */
    /*  $t$ : The timeout */
    /*  $configsMeta$ : meta-data on configurations */
2      $remainingTime \leftarrow t$ 
3      $createdIndexes \leftarrow \emptyset$ 
4      $configsMeta[c].isComplete \leftarrow True$ 
5      $configsMeta[c].indexTime \leftarrow 0$ 

    /* Creates a query-to-indexes map */
6      $I \leftarrow queryIndexMap(Q, c)$ 
    /* Compute the optimal order using Algorithm 4 */
7      $Qsorted \leftarrow FindOptimalOrder(Q, I)$ 

    /* Iterate over queries in optimal order */
8     for  $q \in Qsorted$  do
        /* Create required indexes */
9          $configsMeta[c].indexTime+ = createIndexes(I[q] - createdIndexes)$ 
10         $createdIndexes \leftarrow createdIndexes \cup I[q]$ 
        /* Execute next query */
11         $queryResult \leftarrow Execute(q, remainingTime)$ 
        if not  $queryResult.complete$  then
12             $configsMeta[c].isComplete \leftarrow False$ 
13            break;
        else
14             $remainingTime- = queryResult.executionTime$ 
15             $configsMeta[c].time+ = queryResult.executionTime$ 
16             $configsMeta[c].completedQueries.add(q)$ 
17
19 Function Execute( $q, timeout$ ):
    /* Executes the queries in  $q$  with the given timeout. */
20
    /* Returns a Metrics instance which contains the fields time and completed, indicating
    the execution time and the completion of all queries in  $q$ . */

```

5.3 Optimizing Query Order

To address the problem of optimally ordering query execution and index creation for each configuration evaluation round, we implemented an ordering algorithm based on dynamic programming. Our algorithm is inspired by a classical algorithm for the join ordering problem [16]. Algorithm 4 describes our solution in detail. As input, it takes a set of queries to optimally order, as well as the map I from queries to the set of potentially interesting indexes. The output of Algorithm 4 is the optimal order.

The algorithm is similar to Sellinger’s famous dynamic programming algorithm for join ordering [16]. Similarly to the left-deep or right-deep join ordering problem, given n queries, there are $n!$

possible permutations. However, if the first k queries are sorted, the cost of adding the next query (out of the $n - k$ remaining ones) is independent of the order of the first k ones. More formally, we observe that the principle of optimality holds for our cost function. The principle of optimality states that replacing a solution to a sub-problem with a better solution cannot worsen the overall quality. Specifically, in our case, it means that reducing the expected cost of the first k queries by reordering them, cannot worsen the expected cost of all queries.

THEOREM 5.2. *The principle of optimality holds for expected index creation costs.*

SKETCH. In Equation 1, the cost for the first k queries, scaled by a constant, appears as the first k terms in the outer sum. When calculating the expected cost for k queries, we assume that an interruption after each query has a probability of $1/k$. On the other hand, when considering $n > k$ queries, that probability reduces to $1/n$. However, multiplying the first k terms in the cost function for n queries by factor n/k yields exactly the cost of the first k queries alone. As the cost of the first k queries appears, scaled by a positive constant, as a term in the cost function for n queries, the cost for n queries is monotone in the cost of the first k queries. This means that changing the order of the first k queries to reduce the cost for k queries cannot worsen the overall cost. For the remaining terms (after the k -th term in the outer sum), the order of the first k queries does not matter anymore. $\square$

This insight motivates Algorithm 4. We quickly describe it in the following.

Algorithm 4 maintains the optimal cost for query subsets in Variable *dpCost*. The associated optimal orders are stored in Variable *dbOrder*. The algorithm initializes both data structures in Lines 4 to 7, using single-element query orders and associating singleton queries with the costs of creating the associated indexes.

Next, the algorithm enumerates all subsets of queries in increasing order of set cardinality, starting with query pairs and ending with the set containing all queries. For each subset of queries, it evaluates all possibilities to order the queries when using locally optimal orders for query subsets. More precisely, the algorithm considers all possibilities to expand query orders with k elements into query orders with one additional query. Given a subset of queries for which an optimal order should be calculated, it successively considers each query as a candidate to appear last in the corresponding order. Having chosen a query to appear last, Algorithm 4 retrieves the optimal order for the remaining queries (which must have been calculated in prior iterations since Algorithm 4 considers query sets in ascending order of cardinality) and adds the cost of creating the indexes for the last query. Note that the cost of creating indexes for that last query depends on the indexes that have been already generated for the queries that appear first in the order.

Algorithm 4 updates the best order and associated cost for each of the considered options. Whenever the combined cost of creating indexes for prior queries and creating indexes for the last query is below the best currently known cost for a given query subset, the new order and its cost are stored. Finally, the algorithm returns the best order for the entire query set.

THEOREM 5.3. *Algorithm 4 generates an optimal query order according to our cost model.*

SKETCH. This is a direct consequence of the principle-of-optimality property of our cost function. The algorithm considers all query permutations that use locally optimal solutions for query subsets. However, according to Theorem 5.2, those permutations must contain an optimal solution. $\square$

Note that Algorithm 4 has exponential complexity in the number of input queries as it considers all query subsets. The next section discusses a method by which λ -Tune limits the resulting complexity. This ensures that scheduling does not become a significant time factor during the tuning process.

Algorithm 4: DP Query Scheduling

```

1 Function ComputeOrderDP( $W, I$ ):
    /*  $W$ : The set of queries */
    /*  $I$ : A hashmap containing the indexes for each query */
2    $dpCost \leftarrow \{\}$ 
3    $dpOrder \leftarrow \{\}$ 
4   for  $q \in W$  do
5      $indexes \leftarrow I[q]$ 
6      $dpOrder[\{q\}] \leftarrow [q]$ 
7      $dpTable[\{q\}] \leftarrow cost(indexes)$ 
8   for  $i = 2 \dots n$  do
    /* Enumerate all subsets of size  $i$  */
9     for  $subset \subseteq W : |subset| = i$  do
10       $dpCost[subset] \leftarrow \infty$ 
11      for  $query \in subset$  do
12         $subset' \leftarrow subset - query$ 
13         $queryCost \leftarrow computeCost(query, subset')$ 
14         $c \leftarrow dpTable[subset'] + queryCost$ 
15        if  $c < dpCost[subset]$  then
16           $dpCost[subset] \leftarrow c$ 
17           $dpOrder[subset] \leftarrow dpOrder[subset'] \circ query$ 
18  return  $dpOrder[set(W)]$ 

```

5.4 Query Clustering

Due to the exponential complexity of our query scheduling algorithm, sorting queries for large workloads can be excessively time-consuming. To mitigate this, we reduce the input size to our algorithm by clustering queries according to their index dependencies. To do so, we first need a query vectorization method that complies with standard distance metrics, in order to perform the clustering. Each index is assigned a unique number i , and each query q is represented as a binary vector where the i^{th} element indicates whether q references index i (1 if it does, 0 if it does not). We then cluster the queries based on these vector representations, using the Euclidean distance as the clustering distance metric. Given two queries q and q' , and their index vectors I and I' , then the distance for those two queries can be defined as $d(I, I') = \sqrt{\sum_i^n (I_i - I'_i)^2}$. Using this metric, we can then proceed with clustering the queries using K-Means. This solution is minimally invasive to our algorithm. For instance, consider two queries, $q_1 : A$ and $q_2 : A$, both requiring index A . As long as the algorithm considers only the index creation cost, these queries can be grouped into a single cluster labeled with index A . This approach is also applied to clusters of queries that depend on multiple indexes. We strictly limit the input to our algorithm to a manageable size of 13 queries.

6 Experimental Evaluation

We compare λ -Tune to several baselines on multiple benchmarks and perform an ablation study.

6.1 Experimental Setup

All the experiments were executed on an EC2 p3.2xlarge instance, using the Deep Learning Base GPU AMI on Ubuntu 20.04. As benchmarks, we use the TPC-H benchmark with scaling factors one and ten, TPC-DS with scaling factor one, and the Join Order Benchmark (JOB). We tune Postgres 12.0 and MySQL 8.0. As initial configuration, we use the default settings for all system parameters. Unless noted otherwise, no indexes are initially created.

We compare λ -Tune to two other tuning systems exploiting LLMs, GPTuner [12] and DB-BERT [21]. Also, we compare to UDO [24], a tuning tool for universal database optimization exploiting reinforcement learning. Furthermore, we compare to LlamaTune [9], integrated into the MLOS framework [11], a system that leverages techniques for dimensionality reduction to improve sample efficiency in automated database tuning. For the problem of index selection, we use two specialized tools as baselines, namely Dexter [1] and the DB2 Index Advisor [23]. Finally, we compare to ParamTree [25], a system that tunes five parameters used by the PostgreSQL query optimizer: `cpu_tuple_cost`, `cpu_operator_cost`, `cpu_index_tuple_cost`, `seq_page_cost`, and `random_page_cost`. While ParamTree can optimize settings for those constants on a per-operator level, the PostgreSQL optimizer uses a single value for those parameters for all operators. Hence, for each of the five parameters, we use the average of the operator-specific recommendations. For λ -Tune, we set the timeout for the first round to ten seconds and we set $\alpha = 10$ as well. For all other baselines allowing to set timeouts (namely UDO and GPTuner), to limit their overheads due to bad configurations, we set this timeout to three times the time of the worst configuration found by λ -Tune. UDO executes workload samples to evaluate configurations. Hence, specifically for UDO, we re-execute configurations tried by UDO to measure the execution time for the full workload, thereby making the results comparable to the other baselines. λ -Tune uses OpenAI's GPT-4 model to generate configurations.

6.2 Comparison to Baselines

Table 3. Cost of Best Configuration Found by Each Approach, Scaled to the Cost of the Best Overall Configuration

Benchmark	DBMS	Initial Indexes	λ -Tune	UDO	DB-Bert	GPTuner	LlamaTune	ParamTree
TPC-H 1GB	PG	Yes	1.07	1.96	1.13	1	2.08	3.23
TPC-H 1GB	MS	Yes	1.06	1	1.02	1.73	1.39	3.24
TPC-H 10GB	PG	Yes	1.03	1	1.05	1.04	2.38	3.18
TPC-H 10GB	MS	Yes	4.98	1	5.16	5.84	2.86	15.2
JOB	PG	Yes	1	1.32	1.05	1.1	3.48	3.48
JOB	MS	Yes	1	1.07	3.69	3.69	3.22	3.22
TPC-H 1GB	PG	No	1.05	3.76	1	1.06	1.43	4.24
TPC-H 1GB	MS	No	1.2	2.83	1.02	1	1.61	3.64
TPC-H 10GB	PG	No	1.65	1.54	2.45	2.52	1	1.54
TPC-H 10GB	MS	No	1.04	3.2	1.09	1	1.88	3.2
JOB	PG	No	1	1.69	1.08	1.13	3.09	3.26
JOB	MS	No	1	3.07	3.07	3.07	3.07	3.07
TPC-DS	PG	No	1	1.37	1.67	1.66	3.33	3.33
TPC-DS	MS	No	1.79	3.25	1	1.03	1.05	3.25
Average			1.41	2.00	1.82	1.91	2.27	4.07

Figures 3 and 4 depict the results of our experimental evaluation. In our plots, the x-axis represents the optimization time (in seconds), while the y-axis represents the best execution time found (in

seconds). For instance, the data point (x, y) indicates the best execution time y reported by each framework until time x . All experiments were run three times. For each line plot, the middle line represents the average of the best execution time found over different trials, and the shaded area indicates the error range, encompassing the minimum and maximum execution times found by each configuration. Each line starts at the point in time when the corresponding system had evaluated its first configuration. A dashed line is used in cases where the corresponding system did not evaluate any configurations successfully over the tuning time.

Figure 3 restricts the tuning scope to system parameter tuning. This means tuning approaches cannot change the physical design by creating indexes. Instead, all tuning methods use the same indexes, created before tuning starts and covering primary key and foreign key columns referred to in the input workload. All system parameters are initially set to the default values. Figure 4 expands the tuning scope, allowing baselines to change the physical design as well as settings for system parameters (starting without any indexes and with the default settings for all system parameters). Some of the baselines, namely UDO and λ -Tune, tune parameter settings as well as the physical design. The other baselines focus on system parameters tuning alone. For those systems, we create indexes recommended by Dexter [1] before tuning starts (we omit tuning results for those baselines without Dexter’s indexes as those results are uniformly worse). Table 3 summarizes the results in the two aforementioned figures, reporting the scaled cost of the best configuration found by each baseline. For each scenario, we scale costs to the cost of the optimal configuration found for this scenario by any of the baselines. The “Initial Indexes” column indicates whether indexes are generated before tuning starts (restricting the tuning scope to parameters) or not.

As shown in Table 3, λ -Tune is the most robust tuning method on average, followed by DB-BERT and GPTuner. Interestingly, this means that the three baselines exploiting tuning hints gained from text rank first when averaging over all scenarios. However, DB-BERT and GPTuner still explore a combinatorial search space that combines different hints mined from text. λ -Tune explores a much smaller space, consisting only of the few complete configurations generated by the language model. As demonstrated by the experimental results, this enables λ -Tune to find promising configurations faster. λ -Tune differs from both GPTuner and DB-BERT in that it also recommends indexes (in addition to system parameter settings). Averaging relative costs of the configurations found by λ -Tune in the scenarios allowing index creations only, its relative cost decreases to 1.21 (versus 1.41 when averaging over all scenarios). This shows that λ -Tune’s expanded tuning scope provides additional benefits. UDO is another baseline that optimizes for physical design and parameter settings at the same time. However, lacking hints from text to heuristically guide tuning choices, it converges to optimal decisions more slowly. LlamaTune achieves the best result for TPC-H on PostgreSQL with a scaling factor of 10 and near-optimal results in a few other scenarios. However, LlamaTune does not heuristically constrain the search space via mined tuning hints and suffers from configurations with high run times in some scenarios. LlamaTune has been previously evaluated on OLTP workloads only, using a fixed evaluation time. This scenario makes mechanisms that limit evaluation time spent with bad configurations and OLAP workloads (such as the one implemented by λ -Tune) unnecessary. ParamTree only explores parameter settings with a limited scope (i.e., for the query optimizer), preventing it from tuning other parameters with significant performance benefits.

Table 3 only considers the quality of the best configuration found over the entire tuning time, Figures 4 and 4 provide more details, showing that λ -Tune tends to find optimal configuration also significantly faster, compared to other baselines. This demonstrates the benefits of evaluating complete configurations only.

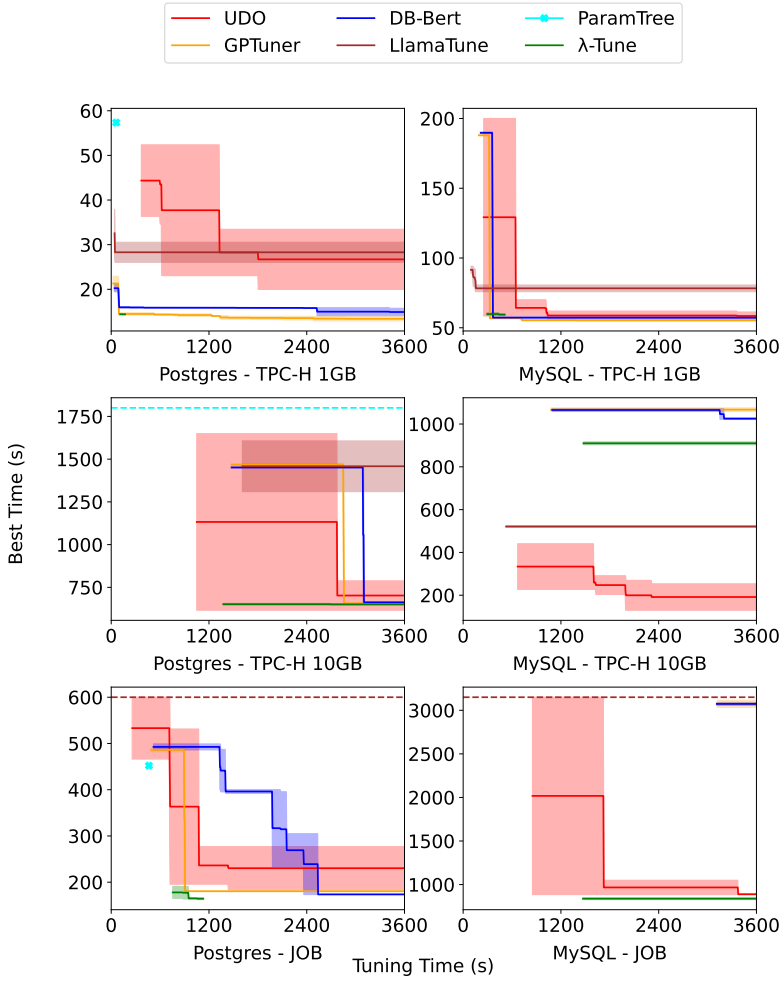


Fig. 3. Scenario 1: Baselines do not Create Indexes (Pure Parameter Tuning), Default Indexes Available

Table 4. Number of Configurations Evaluated per Baseline (Postgres)

Scenario	Initial Indexes	λ -Tune	UDO	DB-Bert	GPTuner	LlamaTune	ParamTree
TPC-H 1GB	Yes	5	617	115	103	10	1
TPC-H 1GB	No	5	707	171	156	19	1
TPC-H 10GB	Yes	5	83	6	3	1	1
TPC-H 10GB	No	5	120	6	5	4	1

6.3 In-Depth Analysis

We analyze the results for one benchmark, TPC-H on Postgres, in more detail. Table 5 shows the configuration selected by λ -Tune in detail. The upper part of Table 5 shows changes to default parameter settings, recommended by λ -Tune. The parameters are classified according to broad categories (e.g., memory and optimizer). Clearly, compared to the default settings, λ -Tune increases

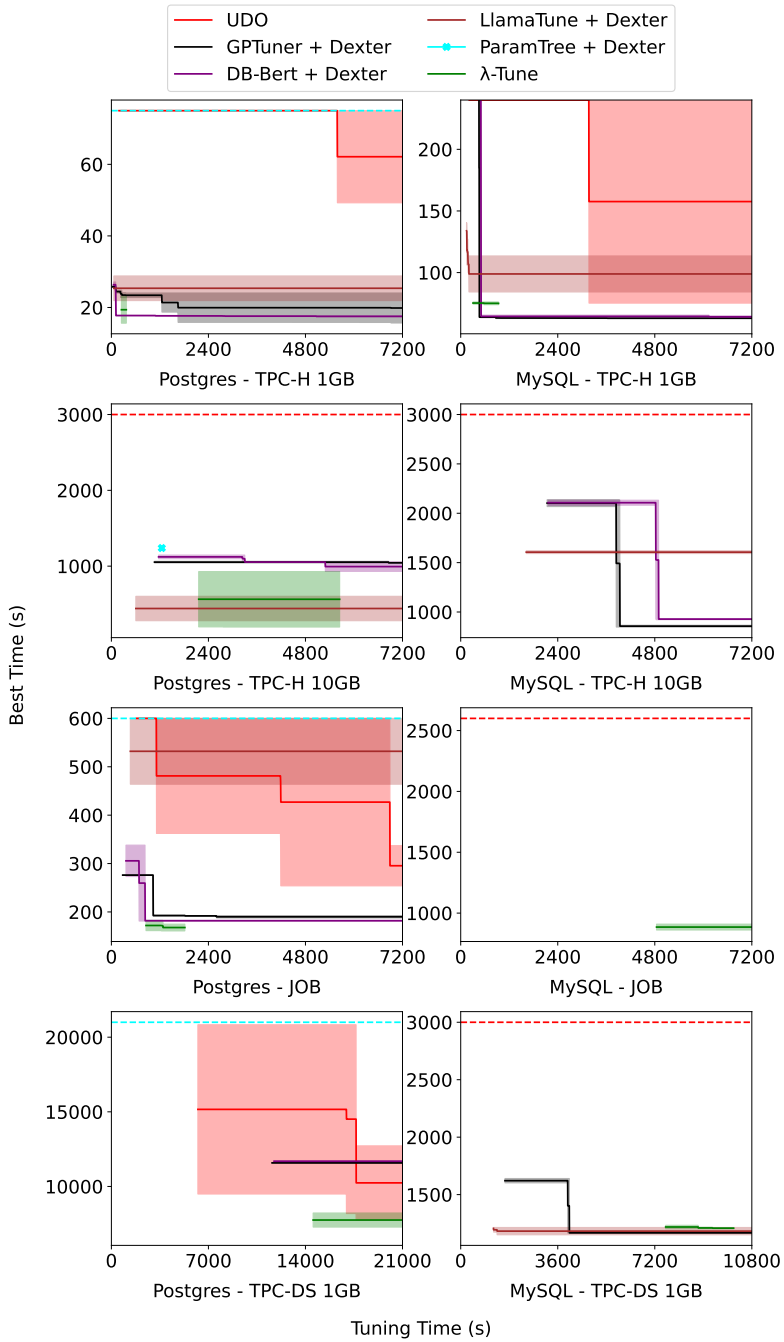
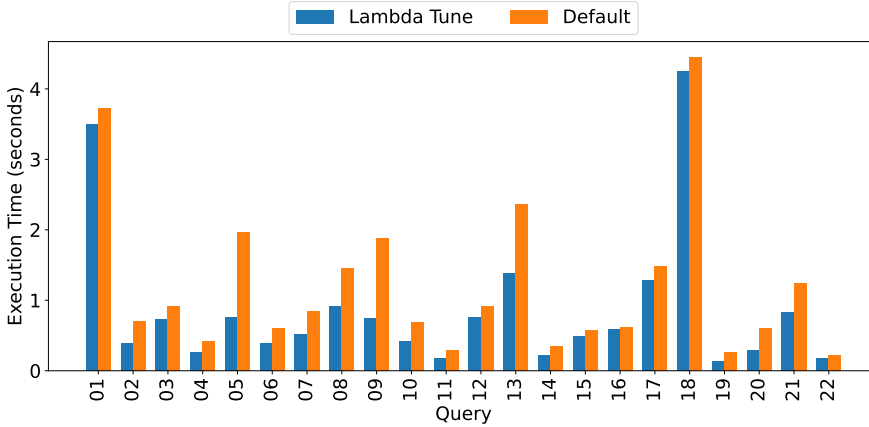


Fig. 4. Scenario 2: Baselines Create Indexes, no Indexes are Created by Default

Table 5. Best λ -Tune Configuration for TPC-H 1GB (Postgres)

Parameter	Category	Value
shared_buffers	Memory	15GB
work_mem	Memory	1GB
effective_cache_size	Optimizer	45GB
maintenance_work_mem	Memory	2GB
checkpoint_completion_target	Logging	0.9
wal_buffers	Logging	16MB
default_statistics_target	Optimizer	100
random_page_cost	Optimizer	1.1
effective_io_concurrency	IO	200

Table	Indexed Columns
lineitem	l_suppkey, l_orderkey, l_partkey
orders	o_custkey, o_orderkey
part	p_partkey
partsupp	ps_partkey, ps_suppkey
supplier	s_nationkey, s_suppkey
customer	c_custkey, c_nationkey
nation	n_nationkey, n_regionkey

Fig. 5. Query Execution Times (TPC-H 1GB, Postgres): λ -Tune vs Default Configuration

the values for several parameters representing the amount of memory reserved for the database management system. This is often beneficial for OLAP workloads. Note that the settings for the `shared_buffers` parameter matches the recommendation from the Postgres manual, stating “a reasonable starting value for `shared_buffers` is 25% of the memory in your system.”. It seems λ -Tune is able to apply this recommendation to the hardware specification of the target system, featuring 61 GB of RAM, as outlined in the hardware description of the input prompt.

Besides memory-related parameters, λ -Tune changes several settings for optimizer-related parameters. In particular, several changes, namely increasing the value of `effective_cache_size` and decreasing the value of `random_page_cost`, compared to the default settings, tend to motivate

the query optimizer of Postgres to use indexes more often. This aligns with the fact that λ -Tune proposes multiple indexes as part of the same recommended configuration.

Also, λ -Tune increases the value of `effective_io_concurrency`, compared to the default setting. This change has been reported to occasionally lead to significant performance improvements for large scans¹. A subset of parameter changes refer to logging behavior which is less relevant for the benchmark, showing that λ -Tune might benefit from additional workload-related information in the prompt.

The lower part of Table 5 reports the indexes created by λ -Tune, reporting the table and the corresponding column (all indexes created by λ -Tune for Postgres were tree indexes with single-column search keys). Clearly, λ -Tune focuses on columns and tables that appear frequently in the query workload.

Index recommendations are generally not transferable between workloads on different databases. However, system parameter settings could, in principle, be transferred between different OLAP workloads. We compare the parameter settings proposed by λ -Tune across the three benchmarks (TPC-H, TPC-DS, and JOB). It turns out that settings for several memory-related parameters tend to be similar (the optimal configurations use the same settings for `shared_buffers` and `maintenance_work_mem`). Also, whenever λ -Tune adds commands for creating indexes, it encourages the optimizer to use them by changing values for `effective_cache_size` and `random_page_cost`. On the other hand, settings for other parameters differ. For instance, only for TPC-DS does λ -Tune choose to increase the `max_parallel_workers_per_gather` parameter value beyond the default. The recommendations generated by the LLM for TPC-H tend to overlap in many aspects (e.g., they consistently set `shared_buffers`) but in 15 LLM samples for the TPC-H prompt, we observe outlier configurations where the run time is up to five times higher than the optimum. This underlines the need for configuration selection with bounded execution time.

Table 4 reports the number of trial runs executed by each baseline. This metric shows significant differences between the different baselines. ParamTree recommends fixed settings for optimizer constants, requiring only a single workload evaluation (after training). λ -Tune evaluates only the five configurations proposed by the LLM. For TPC-H with scaling factor one, this makes λ -Tune one of the most sample-efficient baselines, followed closely by LlamaTune (which increases sample-efficiency via dimensionality reduction). DB-BERT and GPTuner evaluate a significantly higher number of configurations for scaling factor one. UDO evaluates most configurations. However, UDO performs each evaluation with a workload sample (i.e., it does not run all of the queries), meaning that its measurements are not always representative for the quality of a configuration on the full workload. When increasing the scaling factor to 10, the number of trial runs decreases significantly for all baselines except for ParamTree and λ -Tune. This is expected as each run takes longer, due to the increasing input data size.

Finally, we compare per-query execution times between the default settings and the configuration chosen by λ -Tune for TPC-H. Figure 5 reports corresponding results. It turns out that the performance gain via the configuration proposed by λ -Tune translate to gains or at least equal performance, compared to the default settings, for each single query.

6.4 Ablation Study

To showcase the effectiveness of the multiple individual components of λ -Tune, we present the results of an ablation study in which we switched off different components of λ -Tune to measure the resulting performance degradation. The experiments presented next focus on tuning Postgres for the JOB benchmark. Figure 6 depicts the results of our study. The green line (labeled “Default”)

¹https://stborden.wordpress.com/2022/12/27/tuning-the-postgresql-effective_io_concurrency-parameter/

represents the performance of λ -Tune with all of its components enabled. Next, we discuss the impact of specific changes to λ -Tune.

6.4.1 Adaptive Timeout. First, we turn off the component that adaptively sets the timeout of the configuration selection component according to the index creation overheads, discussed in detail in Section 4 (Reconfiguration Overheads). Doing so prevents the system from taking into account reconfiguration overheads when choosing timeouts for query execution. This means that query execution overheads may be dominated by reconfiguration overheads, making the tuning approach inefficient. Indeed, as shown in the plot, this change increases the time needed to find near-optimal configurations from less than 1,000 to over 1,300 seconds. While reconfiguring the system too frequently slows down tuning, it does not decrease the quality of the configurations found by λ -Tune.

6.4.2 Query Scheduler. Next, we showcase the effectiveness of our query scheduler. In turning this component off, we disable λ -Tune’s capability to minimize index creation overheads by optimally ordering queries and creating indexes only if they are immediately relevant. Turning off this component increases the time until the first configuration is completely evaluated (i.e., all queries have finished processing) to more than 1,800 seconds. While this change affects the time needed by λ -Tune to report first evaluation results for its configurations, it does not degrade the quality of the configurations that λ -Tune ultimately returns.

6.4.3 Obfuscated Workload. JOB and TPC-H are popular workloads that are likely to appear in the pre-training data of LLMs such as GPT-4. Hence, apriori, it is unclear whether LLMs generate configurations that appear on the Web. If so, obfuscating the workload should significantly impact λ -Tune’s performance. To test that hypothesis, we hid the names of tables and columns in the input workload. We replaced all table and column names in the extracted query snippets with generic identifiers (e.g., “Tx” and “Cy” where x and y are integer IDs and “T” and “C” indicate tables and columns respectively). It is worth to mention that we obfuscate the snippets after the extraction (we do not provide full queries to the LLM), and thus, it could not imply the benchmark from the query templates as this information is lost after the snippet extraction. The results are reported as “Obfuscated Workload” in Figure 6. However, the results are inconsistent with our hypothesis since the performance remains virtually equivalent to the default settings. This provides evidence that λ -Tune does not benefit from hints in the pre-training data.

6.4.4 Compressor. Describing the input workload by submitting SQL queries as part of the prompt may seem like the most natural approach. Instead, we opt to use a component that compresses the join structure of the input workload. We evaluate performance when switching that compressor off, adding instead as many SQL queries as we can fit into the prompt with the intrinsic token limits of the LLM. For this specific workload, we are able to fit in 26 full SQL queries. However, doing so increases the time until the first configuration is completely evaluated and also increases the execution time of the best configuration found. Furthermore, the number of input tokens, and therefore processing fees due to LLM invocations, increase (this is analyzed in more detail in the plot discussed next).

6.4.5 Token Budget. Figure 7 analyzes the impact of the prompt structure and token limits in more detail. All approaches are labeled with the number of tokens consumed for workload representation (between parentheses). Clearly, adding full SQL queries does not yield optimal performance. However, it is remarkable that λ -Tune is able to generate near-optimal configurations for fairly small token budgets. Only extremely low settings for the token limit (196 tokens) seem to degrade performance significantly. Compared to sending full SQL queries to the LLM, λ -Tune’s compression

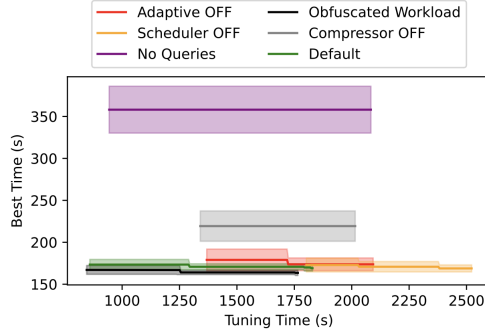


Fig. 6. Ablation - JOB, Postgres, No Indexes

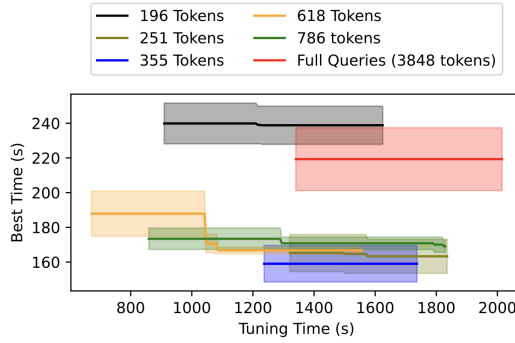


Fig. 7. Ablation - Compressor Budget

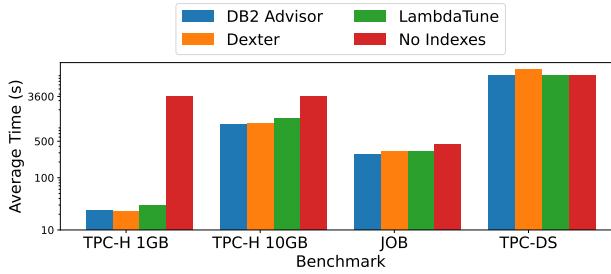


Fig. 8. Comparing Index Recommendation Tools

method achieves better performance even with a token reduction of more than factor ten. These results demonstrate that λ -Tune is able to prioritize effectively which information about the input workload to convey to the LLM.

6.4.6 Index Recommendations. Finally, Figure 8 removes λ -Tune’s ability to change parameter settings, focusing on index recommendation alone. The figure compares the performance of λ -Tune to different index recommendation tools (Dexter and the DB2 Index Advisor), as well as to a configuration without indexes (using the default values for all system parameters). The y-axis

reports execution time and is logarithmic. Using the indexes recommended by λ -Tune reduces run time significantly, compared to the default settings. In most cases, with the exception of TPC-DS, λ -Tune does not achieve the performance of specialized index recommendation tools. This is expected, as λ -Tune has a broader scope.

7 Related Work

λ -Tune leverages recent advances in language models and follows the tradition of learned tuning frameworks. Our related work falls into two categories:

DBMS Tuning. Research efforts have focused on automating various aspects of database systems, such as query optimization [3, 6, 13, 17, 25–28], configuration tuning [2, 8, 10, 14, 24], and index recommendations [5, 9, 23]. Traditional tuning methods [9, 17, 23] are based on cost models, leveraging data statistics to estimate processing overheads with different tuning options. This may lead to sub-optimal tuning choices in case of cost estimation errors. Also, it fails for scenarios such as system parameter tuning where cost models are generally unavailable. Systems such as OtterTune [2], LlamaTune [9], or the ParamTree method [25] use machine learning for database tuning, guided by performance measurements via trial runs. LlamaTune exploits techniques for dimensionality reduction while ParamTree exploits existing cost models to reduce the number of required training samples. Instead, λ -Tune leverages information contained in text documents as a means to reduce the number of required trials in database tuning. λ -Tune’s evaluation component uses timeouts to reduce evaluation overheads. This connects to prior methods used to reduce evaluation overheads, e.g., by selecting subsets of queries [18, 24] or by substituting calls to a classical cost model with invocations of cheaper models [9]. However, λ -Tune reduces evaluation overheads only via timeouts, set in order to guarantee that the system identifies the optimal configuration on the entire workload, out of all configuration generated by the language model.

Large Language Models. Recent work used LLMs for database system performance debugging [19] and tuning [12, 20, 21], with DB-BERT and GPTuner being the most relevant to our work. Both systems use LLMs to extract single hints but must navigate a vast, combinatorial space of tuning hints, requiring numerous trials to find an optimal configuration. λ -Tune becomes more efficient by using the LLM to generate entire configurations. This means we avoid combinatorial search as the LLM already combines hints about different tuning knobs. Also, this approach introduces new challenges, compared to prior work, e.g., in reducing the amount of tokens that are sent to the LLM to describe the tuning context.

8 Conclusion

This paper tested the following hypothesis: *Sampling the output distribution of state-of-the-art LLMs, given a database tuning problem as input, yields at least some efficient configurations.* Our experiments support this hypothesis, even for newly generated workloads that do not appear in the LLM training data. Also, our experiments show that this approach is only effective if LLMs are embedded into a framework that *bounds query evaluation overheads to deal with bad configurations* (via the configuration selection component discussed in Section 4), *minimizes system configuration overheads* (via the smart evaluation component discussed in Section 5), and *limits monetary fees due to LLM invocations* (via the prompt compression approach discussed in Section 3). The resulting system, λ -Tune, outperforms a variety of baselines, including several previously proposed approaches that exploit LLMs for database tuning as well.

Acknowledgments

This material is based upon work supported by the National Science Foundation under Award No. 2239326.

References

- [1] 2021. <https://github.com/ankane/dexter>.
- [2] Dana Van Aken, Andrew Pavlo, and Geoffrey J Gordon. 2017. Automatic database management system tuning through large-scale machine learning. In *SIGMOD*. 1009–1024. doi:10.1145/3035918.3064029
- [3] Christoph Anneser, Nesime Tatbul, David Cohen, Zhenggang Xu, Prithviraj Pandian, Nikolay Laptev, and Ryan Marcus. 2023. AutoSteer: Learned Query Optimization for Any SQL Database. *Proceedings of the VLDB Endowment* 16, 12 (2023), 3515–3527.
- [4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).
- [5] Bailu Ding, Sudipto Das, Ryan Marcus, Wentao Wu, Surajit Chaudhuri, and Vivek R Narasayya. 2019. Ai meets ai: Leveraging query executions to improve index recommendations. In *Proceedings of the 2019 International Conference on Management of Data*. 1241–1258.
- [6] Victor Giannakouris and Immanuel Trummer. 2022. Building Learned Federated Query Optimizers. In *CEUR workshop proceedings*, Vol. 3186.
- [7] Victor Giannakouris and Immanuel Trummer. 2024. Demonstrating λ -Tune: Exploiting Large Language Models for Workload-Adaptive Database System Tuning. In *Companion of the 2024 International Conference on Management of Data*. 508–511.
- [8] Konstantinos Kanellis, Ramnathan Alagappan, and Shivaram Venkataraman. 2020. Too many knobs to tune? towards faster database tuning by pre-selecting important knobs. In *12th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 20)*.
- [9] Konstantinos Kanellis, Cong Ding, Brian Kroth, Andreas Müller, Carlo Curino, and Shivaram Venkataraman. 2022. LlamaTune: Sample-Efficient DBMS Configuration Tuning. *Proceedings of the VLDB Endowment* 15, 11 (2022), 2953–2965. doi:10.14778/3551793.3551844 arXiv:2203.05128
- [10] Konstantinos Kanellis, Cong Ding, Brian Kroth, Andreas Müller, Carlo Curino, and Shivaram Venkataraman. 2022. LlamaTune: Sample-efficient DBMS configuration tuning. *arXiv preprint arXiv:2203.05128* (2022).
- [11] Brian Kroth, Sergiy Matuskevych, Rana Alotaibi, Yiwen Zhu, Anja Gruenheid, and Yuanyuan Tian. 2024. MLOS in Action: Bridging the Gap Between Experimentation and Auto-Tuning in the Cloud. *Proceedings of the VLDB Endowment* 17, 12 (2024), 4269–4272. doi:10.14778/3685800.3685852
- [12] Jiale Lao, Yibo Wang, Yufei Li, Jianping Wang, Yunjia Zhang, Zhiyuan Cheng, Wanghu Chen, Mingjie Tang, and Jianguo Wang. 2023. GPTuner: A Manual-Reading Database Tuning System via GPT-Guided Bayesian Optimization. *arXiv preprint arXiv:2311.03157* (2023).
- [13] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. 2022. Bao: Making learned query optimization practical. *ACM SIGMOD Record* 51, 1 (2022), 6–13.
- [14] Andrew Pavlo, Gustavo Angulo, Joy Arulraj, Haibin Lin, Jiexi Lin, Lin Ma, Prashanth Menon, Todd C Mowry, Matthew Perron, Ian Quah, et al. 2017. Self-Driving Database Management Systems.. In *CIDR*, Vol. 4. 1.
- [15] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. 2018. Improving language understanding by generative pre-training. (2018).
- [16] P Griffiths Selinger, Morton M Astrahan, Donald D Chamberlin, Raymond A Lorie, and Thomas G Price. 1979. Access path selection in a relational database management system. In *Proceedings of the 1979 ACM SIGMOD international conference on Management of data*. 23–34.
- [17] PG G Selinger, MM M Astrahan, D D Chamberlin, R A Lorie, and T G Price. 1979. Access path selection in a relational database management system. In *SIGMOD*. 23–34. <http://dl.acm.org/citation.cfm?id=582095.582099>
- [18] Tarique Siddiqui, Saehan Jo, Wentao Wu, Chi Wang, Vivek Narasayya, and Surajit Chaudhuri. 2022. ISUM: Efficiently Compressing Large and Complex Workloads for Scalable Index Tuning. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 660–673. doi:10.1145/3514221.3526152
- [19] Vikramank Singh, Kapil Eknath Vaidya, Vinayshekhkar Bannihatti Kumar, Sopan Khosla, Murali Narayanaswamy, Rashmi Gangadharaiah, and Tim Kraska. 2024. Panda: Performance debugging for databases using LLM agents. (2024).
- [20] Viraj Thakkar, Madhumitha Sukumar, Jiaxin Dai, Kaushiki Singh, and Zhichao Cao. 2024. Can Modern LLMs Tune and Configure LSM-based Key-Value Stores?. In *Proceedings of the 16th ACM Workshop on Hot Topics in Storage and File Systems*. 116–123.
- [21] Immanuel Trummer. 2022. DB-BERT: a Database Tuning Tool that" Reads the Manual". In *Proceedings of the 2022 International Conference on Management of Data*. 190–203.
- [22] Immanuel Trummer. 2023. Demonstrating GPT-DB: Generating Query-Specific and Customizable Code for SQL Processing with GPT-4. *Proceedings of the VLDB Endowment* 16, 12 (2023), 4098–4101.
- [23] Gary Valentin, Michael Zuliani, Daniel C. Zilio, Guy Lohman, and Alan Skelley. 2000. DB2 Advisor: An optimizer smart enough to recommend its own indexes. In *ICDE*. 101–110. doi:10.1109/icde.2000.839397

- [24] Junxiong Wang, Immanuel Trummer, and Debabrota Basu. 2021. UDO: universal database optimization using reinforcement learning. *arXiv preprint arXiv:2104.01744* (2021).
- [25] Jiani Yang, Sai Wu, Dongxiang Zhang, Jian Dai, Feifei Li, and Gang Chen. 2023. Rethinking Learned Cost Models: Why Start from Scratch? *Proceedings of the ACM on Management of Data* 1, 4 (2023), 1–27. doi:10.1145/3626769
- [26] Zongheng Yang, Wei-Lin Chiang, Sifei Luan, Gautam Mittal, Michael Luo, and Ion Stoica. 2022. Balsa: Learning a Query Optimizer Without Expert Demonstrations. *arXiv preprint arXiv:2201.01441* (2022).
- [27] Yunjia Zhang, Yannis Chronis, Jignesh M Patel, and Theodoros Rekatsinas. 2023. Simple Adaptive Query Processing vs. Learned Query Optimizers: Observations and Analysis. *Proceedings of the VLDB Endowment* 16, 11 (2023), 2962–2975.
- [28] Rong Zhu, Wei Chen, Bolin Ding, Xingguang Chen, Andreas Pfadler, Ziniu Wu, and Jingren Zhou. 2023. Lero: A learning-to-rank query optimizer. *Proceedings of the VLDB Endowment* 16, 6 (2023), 1466–1479.

Received July 2024; revised October 2024; accepted November 2024